

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Направление: 02.03.02 «Фундаментальная информатика и информационные технологии»

ООП: Большие данные и распределенная цифровая платформа

ОТЧЕТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

Тема: Разработка гибридной системы erasure coding и репликации на основе температуры
данных

Выполнила: _____ Добренкова Лия Сергеевна
Учебная группа 22Б15-пу

Научный руководитель: _____ Утешев Алексей Юрьевич
Должность: профессор, заведующий кафедрой
фундаментальной информатики и
распределенных систем
Ученая степень: доктор физико-математических наук

Председатель комиссии: _____ Корхов Владимир Владиславович
Должность: доцент кафедры компьютерного
моделирования и многопроцессорных систем
Ученая степень: кандидат физ.-мат. наук

Санкт-Петербург
2025

СОДЕРЖАНИЕ

1. Введение	3
2. Основы технологий распределенного хранения данных	4
2.1. Репликация: преимущества и недостатки	4
2.2. Кодирование стираний (Erasure Coding)	4
2.2.1. Reed-Solomon (RS)	5
2.2.2. Locally Recoverable Codes (LRC)	5
2.2.3. Сравнение RS и LRC в контексте распределённых систем	6
2.3. Конвертируемые коды (Convertible Codes): оптимизация перекодировки	7
3. Подход Morph	9
3.1 Гибридная схема избыточности	9
Идея гибридной схемы	9
Преимущества гибридной схемы	9
Технические особенности реализации	10
3.2 Конвертируемые коды (Convertible Codes)	10
Технические особенности реализации	11
3.3 Преимущества Morph в управлении I/O и жизненным циклом данных	11
4. Температура данных: концепция и применение	12
4.1. Закон Ципфа и распределение запросов	12
4.2. Классификация данных по температуре: "горячие", "тёплые", "холодные"	12
4.3. Динамическое изменение теплоты данных	13
5. Подход HSM	14
5.1. Управление схемами хранения на основе температуры данных	14
5.2. Учёт глобальной заполняемости дисков	14
5.3. Преимущества и недостатки подхода HSM	15
6. Объединение подходов Morph и HSM	17
6.1. Идея интеграции гибридных схем хранения и температурного анализа	17
6.2. Предложение пайплайна схем хранения	17
6.2.1. Построение последовательности схем хранения	17
6.2.2. Оптимизация диапазонов температур	18
6.3. Преимущества и возможные сложности реализации	18
7. Заключение	19
8. Список использованных источников	20

1. Введение

В последние десятилетия распределённые системы хранения данных становятся неотъемлемой частью ИТ-инфраструктуры в самых разных областях, от облачных вычислений до систем с высокой отказоустойчивостью. Одной из ключевых задач при проектировании таких систем является обеспечение надёжности и доступности данных при минимизации избыточности и затрат на хранение.

Кодирование стираний (Erasure Coding) — это одна из популярных технологий для защиты данных, которая позволяет эффективно восстанавливать утраченные блоки информации. Однако использование стандартных методов кодирования, таких как Reed-Solomon (RS) или Locally Recoverable Codes (LRC), сталкивается с определёнными ограничениями в условиях распределённых систем, таких как высокая стоимость перекодировки или проблемы с управлением I/O.

Целью данного исследования является анализ гибридных подходов к хранению данных, которые объединяют преимущества традиционных методов кодирования стираний с оптимизацией перекодировки и учётом "температуры" данных. В рамках исследования был разработан концептуальный подход, основанный на интеграции гибридных схем хранения с эффективным перекодированием и температурной классификацией данных, что позволяет эффективно управлять жизненным циклом данных и оптимизировать использование ресурсов.

Актуальность работы обусловлена растущими требованиями к масштабируемости, отказоустойчивости и экономической эффективности распределённых систем хранения данных. Разработанные в исследовании методы и подходы могут существенно улучшить производительность и сократить затраты на хранение в современных распределённых системах.

2. Основы технологий распределенного хранения данных

2.1. Репликация: преимущества и недостатки

Репликация — это один из самых простых и широко используемых методов обеспечения отказоустойчивости в распределённых системах. Суть репликации заключается в создании копий данных на нескольких узлах системы, что позволяет обеспечить доступность данных даже в случае выхода из строя части системы. Репликация активно используется в таких системах, как базы данных, файловые хранилища и облачные сервисы.

Преимущества репликации:

- **Повышенная отказоустойчивость.** Копии данных, хранящиеся на различных узлах, обеспечивают доступность информации в случае сбоя одного или нескольких узлов. Это значительно повышает надёжность системы.
- **Ускорение чтения данных.** Наличие нескольких копий данных позволяет распределить нагрузку на систему, ускоряя операции чтения, так как запросы могут быть направлены к любому из репликаторов.
- **Простота реализации.** Репликация является относительно простым и понятным методом защиты данных, её легко внедрить в систему, не требуя сложных математических или инженерных решений.

Недостатки репликации:

- **Высокая избыточность.** Репликация требует хранения нескольких копий одних и тех же данных, что может существенно увеличить объём занимаемого хранилища. Это делает репликацию не всегда экономически эффективным методом в условиях ограниченных ресурсов.
- **Сетевые затраты.** Репликация увеличивает нагрузку на сеть, поскольку для обеспечения актуальности копий требуется передавать данные между узлами, что приводит к дополнительным затратам на передачу данных, особенно в крупных системах.

2.2. Кодирование стираний (Erasure Coding)

Кодирование стираний (Erasure Coding, EC) — это метод защиты данных, при котором исходные данные разделяются на несколько частей и к ним добавляются избыточные данные (паритеты), позволяющие восстановить утраченные блоки. Это эффективный

способ обеспечения надежности данных в распределённых системах, который значительно снижает потребность в избыточном хранении, характерном для репликации.

В рамках кодирования стираний могут применяться различные схемы, среди которых наиболее известными являются **Reed-Solomon (RS)** и **Locally Recoverable Codes (LRC)**.

2.2.1. Reed-Solomon (RS)

Код Reed-Solomon является одним из наиболее распространённых методов кодирования стираний. Он основан на использовании генераторной матрицы и полиномиальных кодов, которые позволяют восстанавливать утраченные данные, используя информацию из других блоков и добавленных паритетов. Это кодирование применяется в различных системах, от RAID-массивов до распределённых облачных хранилищ.

Основные особенности RS:

- **Структура:** В кодах Reed-Solomon данные разбиваются на **k** блоков, к которым добавляются **r** блоков паритета, образуя **EC (Erasure coding) stripe** — полосу данных с избыточной информацией. В случае потери одного или нескольких блоков данных, оставшиеся блоки могут быть использованы для восстановления.
- **Восстановление данных:** При потере данных необходимо обратиться к **k** блокам, включая как данные, так и паритеты, что может увеличить нагрузку на систему, особенно при большом количестве блоков.

Пример схемы Reed-Solomon с **k = 6** и **r = 3**:

- Данные делятся на 6 блоков, и добавляются 3 блоки паритета. При потере до 3 блоков из 9, система может восстановить данные, используя оставшиеся блоки.

2.2.2. Locally Recoverable Codes (LRC)

Locally Recoverable Codes (LRC) — это более специализированный метод кодирования стираний, направленный на повышение эффективности восстановления данных в распределённых системах. LRC используют два типа избыточности: глобальные паритеты для защиты данных на уровне всего страйпа и локальные паритеты, которые позволяют восстанавливать данные с меньшими затратами внутри локальных групп.

Основные особенности LRC:

- **Локальное восстановление:** В отличие от RS, где для восстановления данных необходимо обращаться ко всем блокам, LRC добавляют локальные паритеты для группы данных. Это позволяет восстанавливать утраченные данные, обращаясь только к ограниченному числу блоков при выходе из строя 1 блока, что снижает нагрузку на систему и ускоряет восстановление.
- **Структура:** В LRC создаются два уровня избыточности:
 - **Глобальные паритеты** для восстановления любых блоков при небольших потерях.
 - **Локальные паритеты** для быстрого восстановления данных в пределах каждой группы.

Пример схемы LRC с $k = 12$, $r = 2$ и $l = 2$:

- Данные делятся на 12 блоков, к которым добавляются 2 глобальных паритета и 2 локальных паритета. В случае потери одного блока данных, восстановление сначала производится с использованием локальных паритетов, что ускоряет процесс восстановления и уменьшает нагрузку на систему.
- При потере любых 3 блоков восстановление так же возможно. Кроме того в отдельных случаях можно восстановиться даже при потере 4 блоков. К примеру при потере по 1 блоку в 2 локальных группах и обоих глобальных паритетов.

2.2.3. Сравнение RS и LRC в контексте распределённых систем

Reed-Solomon и **Locally Recoverable Codes** имеют разные характеристики, что определяет их использование в различных сценариях. Reed-Solomon подходит для систем, где важно использовать меньше блоков (например для оптимизации числа I/O) или где вычислительные ресурсы для восстановления данных не являются критическими. Однако его главный недостаток заключается в необходимости обращения ко всем блокам для восстановления данных даже при 1 потере, что может сильно нагрузить систему, особенно при больших объёмах данных.

С другой стороны, **Locally Recoverable Codes** обеспечивают более высокую производительность при восстановлении данных за счёт локализации процесса восстановления и минимизации необходимого объёма данных. Это особенно важно в крупных распределённых системах, где требуется высокая скорость восстановления и снижение нагрузки на систему. Однако LRC могут потребовать более сложной настройки и балансировки по большему числу узлов.

2.3. Конвертируемые коды (Convertible Codes): оптимизация перекодировки

Конвертируемые коды представляют собой метод объединения нескольких кодов в один с целью оптимизации процесса перекодировки данных в распределённых системах. Процесс объединения выполняется в рамках **"merge regime"**, что позволяет преобразовать несколько исходных кодовых слов в одно кодовое слово конечного кода с минимальными затратами на восстановление данных.

Основные принципы и шаги объединения кодов:

1. **Исходные данные:** Для начала имеется несколько исходных кодов, каждый из которых представляет собой код с параметрами $[nI, kI]$, где nI — это общее количество блоков данных, а kI — количество данных в каждом блоке. Каждый исходный код состоит из kI символов данных и $rI = nI - kI$ символов паритета.
2. **Конечный код:** После объединения, конечное кодовое слово создаётся с параметрами $[nF, kF]$, где $kF = \xi kI$, и ξ — целое число ($\xi \geq 2$). Таким образом, количество символов данных увеличивается, чтобы включить данные из всех исходных кодовых слов. Количество символов паритета для конечного кода будет $rF = nF - kF$.
3. **Сохранение данных:** Все данные из исходных кодов сохраняются в неизменённом виде в конечном кодовом слове. Это означает, что $kI \times \xi$ символов данных из исходных кодов остаются неизменными, называемыми **неизменяемыми символами** (unchanged symbols).
4. **Создание новых паритетов:** Паритеты для конечного кода вычисляются как линейные комбинации паритетов из исходных кодов. Например, если для исходных данных a, b, c и паритетов $p1, p2$ были получены паритеты $q1$ и $q2$ в объединённом коде, то:
$$q1 = p1(a, b, c) + p1(d, e, f)$$
$$q2 = p2(a, b, c) + 8 \cdot p2(d, e, f)$$
где d, e, f — это данные из другого исходного кодового слова. Эти новые паритеты помогают обеспечивать устойчивость данных и позволяют восстанавливать утраченные блоки.
5. **Снижение затрат:** Процесс объединения снижает вычислительные затраты, поскольку для вычисления новых паритетов читается только минимально необходимое количество символов из исходных кодов. Например, если $rF < kI$, то для

вычисления новых паритетов достаточно прочитать лишь rF символов из каждого исходного кодового слова, а не все kI , как в традиционных схемах.

6. **Общий результат:** В результате объединения создаётся одно кодовое слово длиной nF , которое включает все данные и паритеты, необходимые для восстановления любых kF символов данных, что гарантирует свойство **MDS (Maximum Distance Separable)**. Это означает, что код обеспечивает максимально возможную степень отказоустойчивости при заданных параметрах.

Преимущества конвертируемых кодов:

- **Эффективность перекодировки:** Конвертируемые коды позволяют объединять несколько исходных кодов в один с минимальными затратами на вычисления и чтение данных, что делает их более эффективными в больших распределенных системах.
- **Гибкость и масштабируемость:** Возможность изменять количество исходных кодов и степень избыточности делает конвертируемые коды подходящими для различных типов распределенных систем.

3. Подход Morph

Morph представляет собой инновационное решение для управления избыточностью данных в распределённых файловых системах, предназначенное для снижения объёма операций ввода-вывода (I/O) при записи и перекодировке файлов. В рамках этого подхода выделяются два ключевых аспекта:

- **Гибридная схема избыточности (Hybrid Redundancy)** для ранних этапов жизненного цикла данных.
- **Конвертируемые коды (Convertible Codes)** для перекодировки между схемами кодирования стираний (EC) на более поздних этапах.

3.1 Гибридная схема избыточности

Идея гибридной схемы

Morph внедряет новый тип схемы избыточности, который сочетает два метода: **репликацию** и **кодирование стираний (EC)**. Данные хранятся одновременно как реплика (или несколько реплик), а также в закодированном виде в **EC-строке (EC stripe)**, что позволяет использовать преимущества обеих технологий. Это решение эффективно снижает нагрузку на систему при работе с "горячими" данными и ускоряет процесс записи.

Пример гибридной схемы: $Nu(1, EC(6,9))$ — данные представлены одной репликой и одной EC-строкой, содержащей 6 блоков данных и 3 блока паритета. В этом случае достигается баланс между высокой доступностью (через реплику) и надёжностью (через EC).

Преимущества гибридной схемы

1. **Снижение нагрузки на ввод/вывод (I/O):** Гибридная схема требует меньшей избыточности по сравнению с традиционными схемами, такими как трёхкратная репликация (например, 150% вместо 200% в случае с тремя репликами). При переходе от гибридной схемы к чистой EC-схеме реплика удаляется без необходимости считывания или записи данных.
2. **Сохранение производительности:** Для "горячих" данных, где важно минимизировать задержки при записи и чтении, гибридная схема обеспечивает

такую же низкую задержку, как и трёхкратная репликация. Реплики используются для быстрого доступа, в то время как паритеты ЕС вычисляются асинхронно.

3. **Простота перехода:** Переход из гибридной схемы в чистую ЕС-схему осуществляется путём простого изменения метаданных и удаления реплики, что делает этот процесс не ресурсоёмким и быстрым.

Технические особенности реализации

1. Запись в гибридной схеме (Hybrid Writes):

- Для небольших файлов, которые можно разместить в одном блоке ЕС, клиент записывает данные в реплику и один блок ЕС. После подтверждения записи данные считаются устойчивыми. Остальные ЕС-паритеты вычисляются асинхронно.
- Для больших файлов, которые записываются на несколько блоков ЕС, данные сначала записываются в три реплики для обеспечения низкой задержки, а затем данные преобразуются в ЕС-строку и паритеты записываются на соответствующие серверы.

2. Чтение в гибридной схеме (Hybrid Reads):

- **Чтение с реплики:** Для быстрых запросов данные читаются с реплики с минимальной задержкой.
- **Полосное чтение (Striped Reads):** Для крупных запросов данные читаются параллельно с нескольких блоков ЕС, что увеличивает пропускную способность.
- **Чтение в режиме деградации:** Если один из блоков данных недоступен, система использует реплику или восстанавливает данные из оставшихся блоков ЕС.

3. **Преобразование из гибридной схемы в ЕС-схему:** Переход осуществляется удалением реплики, что не требует дополнительных операций ввода-вывода и существенно упрощает процесс.

3.2 Конвертируемые коды (Convertible Codes)

Конвертируемые коды (Convertible Codes, CC) — в morph есть нативная возможность DFS использования конвертируемых кодов с помощью merge regime, но также поддерживается и split для обратного перехода. Кроме того есть возможность повышать абсолютное количество избыточных блоков.

Пример: при переходе с EC(6,9) на EC(12,15) Morph считывает только 6 паритетных блоков, а не 12 блоков данных, что сокращает количество необходимых операций ввода-вывода.

Технические особенности реализации

1. **Выбор параметров ЕС для оптимальной перекодировки:** Morph использует специальные эвристики для выбора параметров ЕС-схем, чтобы максимизировать преимущества конвертируемых кодов:
 - Ширина конечного EC-stripe должна быть кратна ширине исходного stripe.
 - Количество паритетов в новой схеме не должно увеличиваться, для оптимальных операций перекодирования.
2. **Оптимизация размещения данных:** Для избежания перемещения блоков данных в процессе перекодировки Morph заранее планирует размещение блоков данных и паритетов, что позволяет формировать будущие ЕС-строки без необходимости перемещать данные.
3. **Поддержка перекодировки в DFS:** Morph реализует перекодировку как нативную операцию в файловой системе, включая формирование новых ЕС-строк, вычисление новых паритетов и обновление метаданных.

3.3 Преимущества Morph в управлении I/O и жизненным циклом данных

Morph предлагает эффективное решение для управления избыточностью данных, комбинируя гибридные схемы избыточности для ранних этапов жизненного цикла данных с конвертируемыми кодами для более поздних этапов. Эти подходы позволяют существенно сократить объём операций ввода-вывода, снизив нагрузку на диски и сеть.

4. Температура данных: концепция и применение

4.1. Закон Ципфа и распределение запросов

Закон Ципфа - эмпирическая закономерность распределения частотности слов естественного языка: если все слова языка (или просто достаточно длинного текста) упорядочить по убыванию частотности их использования, то частотность n -го слова в таком списке окажется приблизительно обратно пропорциональной его порядковому номеру n .

Это же наблюдается и в частотности обращения к данным. Это распределение имеет важные последствия для организации хранения данных, поскольку позволяет выделить наиболее востребованные данные ("горячие"), которые требуют особого внимания в плане производительности и ресурсов, и менее востребованные данные ("холодные"), которые можно хранить с меньшими затратами.

4.2. Классификация данных по температуре: "горячие", "тёплые", "холодные"

В контексте системы хранения данных данные классифицируются по их "температуре" на несколько категорий в зависимости от частоты доступа:

- **Горячие данные:** Это данные, к которым осуществляется наибольшее количество запросов. Обычно они составляют около 20% от общего объема данных, но генерируют 80% всех запросов. Эти данные требуют высокой производительности и часто хранятся с использованием репликации или других технологий, оптимизированных для быстрого доступа.
- **Тёплые данные:** Данные, к которым запросы поступают с умеренной частотой. Такие данные могут составлять 20-30% от общего объема, и их нужно хранить с балансом между производительностью и экономией места. В некоторых случаях для "тёплых" данных применяются менее ресурсоемкие методы, такие как сжатие или более низкая степень репликации.
- **Холодные данные:** Это данные, к которым запрашиваются редко, и которые могут составлять до 80% всех данных в системе. Поскольку к ним обращаются очень редко, для хранения таких данных предпочтительнее использовать схемы, которые минимизируют затраты на хранение, такие как кодирование стираний (EC) или архивирование.

- **Замороженные:** Архивные данные, которые зачастую переписываются на специальное оборудование. Планируется крайне редкое использование.

4.3. Динамическое изменение теплоты данных

Температура данных не является постоянной характеристикой. Данные, которые были горячими, могут со временем стать менее популярными, а соответственно — холодными. В обратную сторону процесс также возможен: "холодные" данные могут снова стать востребованными, становясь "горячими".

Для эффективного управления такими изменениями разработаны различные алгоритмы, которые отслеживают частоту запросов к данным. Эти алгоритмы позволяют перенаправлять данные между различными уровнями хранения в зависимости от их текущей популярности, например, переводить "горячие" данные на более быстрые и дорогие хранилища или архивировать "холодные" данные, перемещая их на более экономичные носители.

5. Подход HSM

5.1. Управление схемами хранения на основе температуры данных

В методе HSM (Hybrid Storage Method) управление схемами хранения данных основывается на их температуре, что позволяет динамически адаптировать стратегию хранения в зависимости от частоты доступа к данным. В этом подходе данные классифицируются на «горячие» и «холодные», исходя из того, как часто они запрашиваются.

Температура данных в методе HSM определяется на основе частоты их доступа и отражает их популярность. Для этого используется закон Ципфа.

- **Горячие данные (hot data):** 20% данных с самым высоким значением температуры, которые составляют основную часть запросов (80%).
- **Холодные данные (cold data):** 80% данных с низким значением температуры, которые реже запрашиваются.

Температура данных вычисляется по следующей формуле:

$$H = N/T$$

где:

- H — показатель температуры данных.
- N — количество запросов к данным за определённый временной интервал.
- T — временной интервал.

Данные с наибольшим значением температуры (в верхних 20%) считаются «горячими», и для них применяется трёхкратная репликация. Данные с низким значением температуры (в нижних 80%) классифицируются как «холодные», для которых применяются более экономичные схемы хранения, такие как двухкратная репликация или кодирование стираний.

5.2. Учёт глобальной заполняемости дисков

В зависимости от текущего уровня заполненности дискового пространства система адаптирует стратегию хранения данных, чтобы сбалансировать производительность и экономию места.

1. **Низкая заполняемость ($\leq 30\%$):** При низкой заполняемости пространство достаточно для хранения всех данных с трёхкратной репликацией, что обеспечивает максимальную скорость чтения для «горячих» данных.
2. **Средняя заполняемость (30-60%):** При средней заполняемости часть данных (особенно «холодные») переводятся на двухкратную репликацию для уменьшения затрат на хранение, сохраняя при этом трёхкратную репликацию для наиболее востребованных «горячих» данных.
3. **Высокая заполняемость ($> 60\%$):** При высокой заполняемости дисков система минимизирует затраты на хранение, уменьшая количество реплик для «горячих» данных и архивируя «холодные» данные с использованием ЕС.

5.3. Преимущества и недостатки подхода HSM

Преимущества:

1. **Учет температуры данных:** Метод HSM позволяет динамически перераспределять данные между «горячими» и «холодными», используя различные методы хранения для каждого типа данных. Это способствует более эффективному использованию доступных дисков и снижению затрат на хранение.
2. **Адаптивность к системным ресурсам:** Система автоматически адаптируется к изменению заполняемости дисков, что помогает поддерживать баланс между производительностью и затратами без необходимости вмешательства администратора.
3. **Оптимизация времени отклика:** Для «горячих» данных используется трёхкратная репликация, что гарантирует быстрый доступ к этим данным, а для «холодных» применяется ЕС, что снижает затраты на хранение без значительного влияния на производительность.

Недостатки:

1. **Мало категорий температур:** Реализация системы HSM учитывает только 2 категории, что может разделить схожие данные на совершенно разные способы хранения.
2. **Время на перекодирование:** Для «холодных» данных, хранящихся с использованием кодирования стираний, требуется время на перекодировку. Так как не используется Ну-схема из morph, данные будут заблокированы на время перекодирования.

3. **Высокое потребление памяти:** Почти все модели основаны на репликах, что дает очень высокую избыточность. Несмотря на наличие ЕС в худшем случае для холодных данных, дисковое пространство можно было бы использовать эффективнее

6. Объединение подходов Morph и HSM

6.1. Идея интеграции гибридных схем хранения и температурного анализа

Интеграция подходов Morph и HSM предполагает создание более гибкой и адаптивной системы управления хранением данных, которая сочетает в себе лучшие стороны обоих методов. В HSM данные классифицируются по температуре, а в Morph применяется динамическое перераспределение данных между различными схемами хранения.

Объединение этих подходов позволяет реализовать более точное и гибкое управление данными, учитывающее как частоту их запросов, так и характеристики различных схем хранения.

Идея интеграции заключается в автоматическом переходе данных между схемами хранения на основе их температуры. В процессе работы система отслеживает, как меняется температура данных, и на основе этих данных оптимизирует выбор схемы хранения, учитывая как стоимость хранения, так и требования к производительности. Это позволит системе более точно управлять хранением, минимизируя затраты и улучшая время отклика на запросы.

6.2. Предложение пайплайна схем хранения

Для реализации интеграции подходов Morph и HSM необходимо построить пайплайн схем хранения, который будет учитывать как температуру данных, так и текущее состояние дисков, включая их заполненность. Этот пайплайн будет включать последовательность схем хранения, каждая из которых будет соответствовать определённому диапазону температур и характеристикам данных.

6.2.1. Построение последовательности схем хранения

Последовательность схем хранения должна быть выстроена так, чтобы обеспечить наилучшую производительность при минимальных затратах на хранение данных. Каждая схема будет применяться в зависимости от температуры данных и заполняемости дисков. В morph мы могли определить оптимальные пары схем для перехода из одной в другую по кратности EC-stripe. Поэтому можно заранее выбрать последовательность схем с оптимальной перекодировкой, например вот такой пайплайн:

Hy(1, RS(6,9)) -> RS(6, 9) -> LRC(12, 2, 2) -> LRC(24, 4, 2)

6.2.2. Оптимизация диапазонов температур

По любой паре схем возможно сказать во сколько раз должно быть меньше I/O операций, чтобы нагрузка, создаваемая конкретными данными, осталась прежней. По данным можно выбрать таймфрейм, в котором будет рассчитываться реальные I/O и сопоставляться с нужной схемой хранения. Например, при модификации 1 чанка данных в схеме RS(6, 9) потребуется записать 1 чанк данных, прочесть оставшиеся 5, записать 3 новых parity чанков = 9 I/O. Для схемы LRC(12, 2, 2) - 1 запись чанка данных, чтение оставшихся 11 чанков, обновление 2 global parity чанков, обновление 1 local parity чанка = 15 I/O. Соответственно, если температура данных упала сильнее, чем в 15/9 раз, то имеет смысл перекодировать в новую схему, тк нагрузка I/O на запись не увеличиться.

6.3. Преимущества и возможные сложности реализации

Преимущества:

1. **Оптимизация производительности и затрат:** Объединение подходов Morph и HSM позволяет оптимизировать переходы между схемами хранения в зависимости от температуры данных. Это снижает затраты на хранение данных.
2. **Автоматизация процессов:** Создание пайплайна схем хранения и вычисление оптимальных диапазонов температур позволяет автоматизировать управление данными, минимизируя необходимость в вмешательстве оператора.
3. **Гибкость:** Пайплайн схем хранения позволяет гибко реагировать на изменения в потребностях системы, таких как изменение температуры данных, что делает систему более адаптивной.
4. **Оптимальные перекодировки:** Благодаря morph, можно использовать больше температурных категорий и перекодировать данные тогда, когда это действительно требуется.

7. Заключение

В ходе работы:

- 1) Были изучены основные концепции обеспечения отказоустойчивости в распределенных хранилищах
- 2) Рассмотрены гибридные системы, их преимущества, недостатки
- 3) Предложена идея гибридной системы, потенциально объединяющей плюсы рассмотренных систем.

8. Список использованных источников

- [1] Kim T. et al. Morph: Efficient File-Lifetime Redundancy Management for Cluster File Systems //Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles. – 2024. – С. 330-346.
- [2] Song Y. et al. HSM: A hybrid storage method based on the heat of data and global disk space utilization //IEEE Access. – 2024.
- [3] Maturana F., Rashmi K. V. Convertible codes: Enabling efficient conversion of coded data in distributed storage //IEEE Transactions on Information Theory. – 2022. – Т. 68. – №. 7. – С. 4392-4407.
- [4] Noor J. et al. Towards benchmarking erasure coding schemes in object storage system: A systematic review //Future Generation Computer Systems. – 2024. – С. 107522.
- [5] Шинкаренко К. В. и др. Помехоустойчивое кодирование мультимедиа данных в компьютерных сетях //Известия Томского политехнического университета. Инжиниринг георесурсов. – 2008. – Т. 313. – №. 5. – С. 37-41.
- [6] Adamic L. A. Zipf, power-laws, and pareto-a ranking tutorial //Xerox Palo Alto Research Center, Palo Alto, CA, <http://ginger.hpl.hp.com/shl/papers/ranking/ranking.html>. – 2000.
- [7] Wei B. et al. Erasure-Coded Hybrid Writes Based on Data Delta //International Journal of Parallel Programming. – 2024. – С. 1-22.
- [8] Maturana F., Rashmi K. V. Locally repairable convertible codes: Erasure codes for efficient repair and conversion //2023 IEEE International Symposium on Information Theory (ISIT). – IEEE, 2023. – С. 2033-2038.
- [9] Apache Hadoop Y. Available online: <https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/YARN.html> (accessed on 23 January 2024). – 2024.
- [10] Maturana F., Rashmi K. V. Bandwidth cost of code conversions in distributed storage: Fundamental limits and optimal constructions //IEEE Transactions on Information Theory. – 2023. – Т. 69. – №. 8. – С. 4993-5008.